答疑汇总(thread,async源码分析)

Posted on 2023-10-19 15:50:44 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2WyPSKn5SVHAMVwFrQkCnyGn1kF)

简介

本文件汇总粉丝提出的关于并发的几个问题，做个备份，方便大家理解和学习。

局部变量返回值

关于局部变量返回值的问题我曾在视频中说会通过构造函数返回一个局部变量给调用者，编译器会先执行拷贝构造，如果没有拷贝构造再寻找移动构造。这么说是有问题的。
有热心的粉丝查询了chatgpt，当函数返回一个类类型的局部变量时会先调用移动构造，如果没有移动构造再调用拷贝构造。
所以对于一些没有拷贝构造但是实现了移动构造的类类型也支持通过函数返回局部变量。
在 C++11 之后，编译器会默认使用移动语义（move semantics）来提高性能

看个例子

```
1. class TestCopy {
2. public:
3.     TestCopy(){}
4.     TestCopy(const TestCopy& tp) {
5.         std::cout << "Test Copy Copy " << std::endl;
6.     }
7.     TestCopy(TestCopy&& cp) {
8.         std::cout << "Test Copy Move " << std::endl;
9.     }
10. };
11.
12. TestCopy TestCp() {
13.     TestCopy tp;
14.     return tp;
15. }
```

main 函数中调用TestCp

```
int main(){
    TestCp();
    return 0;
}
```

发现打印的是”Test Copy Move” .这说明优先调用的是移动构造，这也提醒我们，如果我们自定义的类实现了拷贝构造和移动构造，而这个类的移动给构造和拷贝构造实现方式不同时，要注意通过函数内部局部变量返回该类时调用移动构造是否会存在一些逻辑或安全的问题。

优先按照移动构造的方式返回局部的类对象，有一个好处就是可以返回一些只支持移动构造的类型

```
std::unique_ptr<int> ReturnUniquePtr() {
    std::unique_ptr<int>  uq_ptr = std::make_unique<int>(100);
    return  uq_ptr;
}

std::thread ReturnThread() {
    std::thread t([]() {
        int i = 0;
        while (true) {
            std::cout << "i is " << i << std::endl;
            i++;
            if (i == 5) {
                break;
            }
            std::this_thread::sleep_for(std::chrono::seconds(1));
        }
        });

    return t;
}
```

main函数中调用后，可以看到线程和 `unique_ptr` 都可被函数作为局部变量返回，而且返回的线程可以继续运行。

```
int main(){
    auto rt_ptr = ReturnUniquePtr();
    std::cout << "rt_ptr value is " << *rt_ptr << std::endl;
    std::thread rt_thread = ReturnThread();
    rt_thread.join();
    return 0;
}
```

线程归属权问题

有粉丝反馈在使用thread时遇到崩溃，主要原因在于线程归属权没有理解好，我们不能将一个线程的归属权转移给一个已经绑定线程的变量。

比如下面的调用

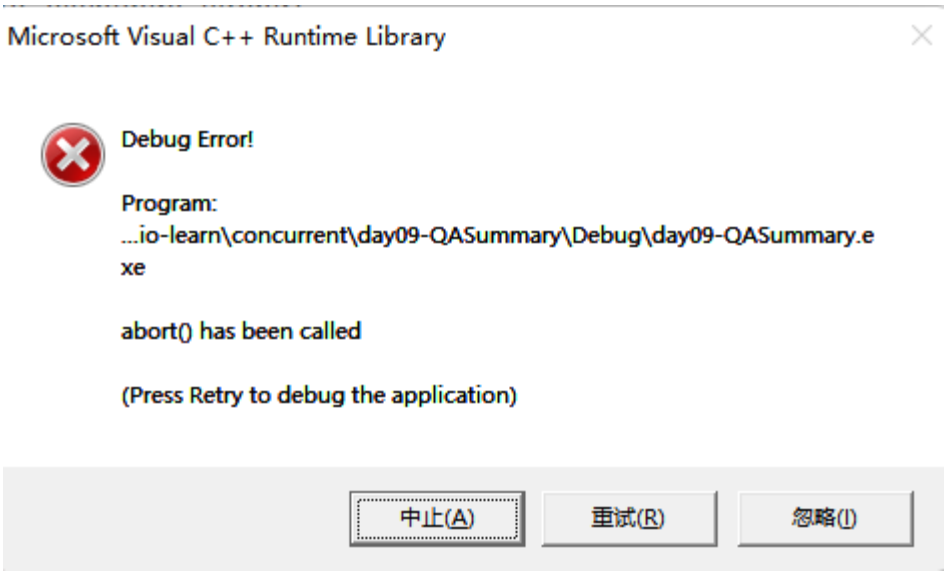
```
void ThreadOp() {

    std::thread t1([]() {
        int i = 0;
        while (i < 5) {
            std::this_thread::sleep_for(std::chrono::seconds(1));
            i++;
            }
        });

    std::thread t2([]() {
        int i = 0;
        while (i < 10) {
            std::this_thread::sleep_for(std::chrono::seconds(1));
            i++;
        }
        });

    //不能将一个线程归属权绑定给一个已经绑定线程的变量，否则会触发terminate导致崩溃
    t1 = std::move(t2);
    t1.join();
    t2.join();
}
```

我们在主函数中执行上述函数，会触发崩溃如下图



`t1` 已经绑定了一个线程执行循环操作直到 `i<5` 。如果在 `t1` 没运行完的情况下将 `t2` 的归属权给 `t1` ，则会引发terminate崩溃错误。

具体原因我们可以看看 `thread` 在做移动赋值时的源码

```
1. thread& operator=(thread&& _Other) noexcept {
2.     if (joinable()) {
3.         _STD terminate();
4.     }
5.
6.     _Thr = _STD exchange(_Other._Thr, {});
7.     return *this;
8. }
```

在线程 `joinable()` 返回 `true` 时，会触发 `terminate()` 操作，也就是被赋值的线程没有被 `join` 过，此时执行 `operator=` 操作会导致 `terminate()` 。
至于 `terminate()` 实现比较简单

```
1.  _ACRTIMP __declspec(noreturn) void __cdecl terminate() throw();
```

可以看到 `terminate()` 就是抛出异常。

所以我们在之前的课程封装了了自动join的线程类

```
class joining_thread {
    std::thread  _t;
public:
    joining_thread() noexcept = default;
    template<typename Callable, typename ...  Args>
    explicit  joining_thread(Callable&& func, Args&& ...args):
        t(std::forward<Callable>(func),  std::forward<Args>(args)...){}
    explicit joining_thread(std::thread  t) noexcept: _t(std::move(t)){}
    joining_thread(joining_thread&& other) noexcept: _t(std::move(other._t)){}
    joining_thread& operator=(joining_thread&& other) noexcept
    {
        //如果当前线程可汇合，则汇合等待线程完成再赋值
        if (joinable()) {
            join();
        }
        _t = std::move(other._t);
        return *this;
    }

    joining_thread& operator=(joining_thread other) noexcept
    {
        //如果当前线程可汇合，则汇合等待线程完成再赋值
        if (joinable()) {
            join();
        }
        _t = std::move(other._t);
        return *this;
    }


    ~joining_thread() noexcept {
        if (joinable()) {
            join();
        }
    }

    void swap(joining_thread& other) noexcept {
        _t.swap(other._t);
    }

    std::thread::id   get_id() const noexcept {
        return _t.get_id();
```

```
    }

    bool joinable() const noexcept {
        return _t.joinable();
    }

    void join() {
        _t.join();
    }

    void detach() {
        _t.detach();
    }

    std::thread& as_thread() noexcept {
        return _t;
    }

    const std::thread& as_thread() const noexcept {
        return _t;
    }
};
```

thread参数值拷贝

之前讲到构造 `std::thread` 对象传递回调函数和参数，回调函数的参数绑定都是值拷贝的方式，这里再梳理一次
下面是 `thread` 的构造函数

```
template <class _Fn, class... _Args, enable_if_t<!is_same_v<_Remove_cvref_t<_Fn>, thread>, int> = 0>
    _NODISCARD_CTOR explicit thread(_Fn&& _Fx, _Args&&... _Ax) {
        _Start(_STD forward<_Fn>(_Fx), _STD forward<_Args>(_Ax)...);
    }
```

构造函数内调用了 `_Start` 函数

```
template <class _Fn, class... _Args>
void _Start(_Fn&& _Fx, _Args&&... _Ax) {
    // 1 处
    using _Tuple              = tuple<decay_t<_Fn>, decay_t<_Args>...>;
    // 2 处
    auto _Decay_copied        = _STD make_unique<_Tuple>(_STD forward<_Fn>(_Fx), _STD forward<_Args>(_Ax)...);
    // 3 处
    constexpr auto _Invoker_proc = _Get_invoke<_Tuple>(make_index_sequence<1 + sizeof...(_Args)>{});

#pragma warning(push)
#pragma warning(disable : 5039) // pointer or reference to potentially throwing function passed to
                                // extern C function under -EHc. Undefined behavior may occur
                                // if this function throws an exception. (/Wall)
    // 4处
    _Thr._Hnd =
        reinterpret_cast<void*>(_CSTD _beginthreadex(nullptr, 0, _Invoker_proc, _Decay_copied.get(), 0, &_Thr._Id));
#pragma warning(pop)

    if (_Thr._Hnd) { // ownership transferred to the thread
        (void) _Decay_copied.release();
    } else { // failed to start thread
        _Thr._Id = 0;
        _Throw_Cpp_error(_RESOURCE_UNAVAILABLE_TRY_AGAIN);
    }
}
```

我们从上面的代码 1处 可以看到 `_Tuple` 是一个去引用的类型，因为其内部存储的都是 `decay_t` 过后的类型，所以无论左值引用还是右值引用到这里都变为去引用的类型。

所以2处就是将参数和函数按照值拷贝的方式存在tuple中。

3处定义了一个可调用对象 `_Invoker_proc`

4处启动线程调用 `_Invoker_proc` 进而调用我们传递的回调函数和参数。

所以综上所述， `std::thread` 向回调函数传递值是以副本的方式,回调函数参数是引用类型，可以将传递的实参用 `std::ref` 包装达到修改的效果。
因为 `std::ref` 其实是构造了 `reference_wrapper` 类对象，这个类实现了仿函数

```
1. _CONSTEXPR20 operator _Ty&() const noexcept {
2.         return *_Ptr;
3. }
```

所以当线程接收 `std::ref` 包裹的参数时会调用仿函数通过指针解引用的方式获取外部实参，以 `_Ty&` 返回，从而达到修改的效果。

那么如下调用就会报错，提示“invoke”: 未找到匹配的重载函数。

```
1. void ChangeValue() {
2.     int m = 100;
3.     std::thread  t1{ [](int& rm) {
4.         rm++;
5.     }, m };
6.
7.     t1.join();
8. }
```

因为 `invoke` 函数调用时会将参数以右值的方式移动给回调函数，这会造成左值引用绑定右值的情况，所以编译报错。

改为下面这样写就没问题了

```
1. void ChangeValue() {
2.     int m = 100;
3.     std::thread  t1{ [](int& rm) {
4.         rm++;
5.     }, std::ref(m) };
6.
7.     t1.join();
8. }
```

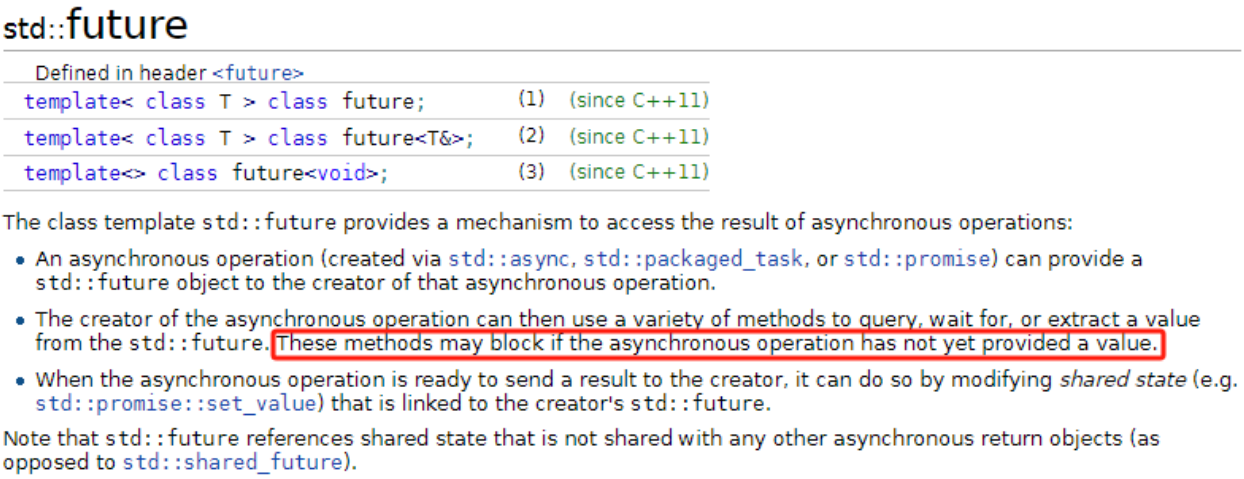
async注意事项

部分粉丝反馈async不能像js那样实现完全的纯异步，确实是存在这样的情况，因为于js不同，js是单线程的，而C++需要关注线程的生命周期。

我们使用 `async` 时，其实其内部调用了 `thread` ，`pacakged_task` ，`future` 等机制。 `async` 会返回一个 `future` 这个 `future` 如果会在被析构时等待其绑定的线程任务是否

执行完成。

我们看一段 cppreference.com 中的描述



std::future

Defined in header <future>

```
template< class T > class future;          (1)  (since C++11)
template< class T > class future<T&>;      (2)  (since C++11)
template<> class future<void>;             (3)  (since C++11)
```

The class template std::future provides a mechanism to access the result of asynchronous operations:

- An asynchronous operation (created via std::async, std::packaged_task, or std::promise) can provide a std::future object to the creator of that asynchronous operation.
- The creator of the asynchronous operation can then use a variety of methods to query, wait for, or extract a value from the std::future. These methods may block if the asynchronous operation has not yet provided a value.
- When the asynchronous operation is ready to send a result to the creator, it can do so by modifying *shared state* (e.g. std::promise::set_value) that is linked to the creator's std::future.

Note that std::future references shared state that is not shared with any other asynchronous return objects (as opposed to std::shared_future).

“The creator of the asynchronous operation can then use a variety of methods to query, wait for, or extract a value from the std::future. These methods may block if the asynchronous operation has not yet provided a value.”

异步操作 async 返回 std::future , 调用者可以通过 query , wait for 等方式
从 std::future 中查询状态。
但是如果 async 直接调用而不适用返回值则可能会阻塞。如下例子

```
1. void BlockAsync() {
2.     std::cout << "begin block async" << std::endl;
3.     {
4.         std::async(std::launch::async, []() {
5.             std::this_thread::sleep_for(std::chrono::seconds(3));
6.             std::cout << "std::async called " << std::endl;
7.             });
8.     }
9.     std::cout << "end block async" << std::endl;
10. }
```

我们在主函数调用 BlockAsync() ， 发现 async 并没有异步执行任务，而是按次序输出如下

```
1. begin block async
2. std::async called
3. end block async
```

因为 `async` 返回一个右值类型的 `future` ，无论左值还是右值， `future` 都要被析构，因为其处于一个局部作用域 `{}` 中。
当编译器执行到 `}` 时会触发 `future` 析构。但是 `future` 析构要保证其关联的任务完成，所以需要等待任务完成 `future` 才被析构，
所以也就成了串行的效果了。

所以C++ 官方文档说 如果调用析构函数的那个future是某一shared state的最后持有者，而相关的task已启动但尚未结束，析构函数会造成阻塞，直到任务结束

至于为什么 `future` 析构要等到其关联的任务完成我们可以看一下 `async` 源码

```
1. template <class _Fty, class... _ArgTypes>
2. _NODISCARD future<_Invoke_result_t<decay_t<_Fty>, decay_t<_ArgTypes>...>> async(
3.     launch _Policy, _Fty&& _Fnarg, _ArgTypes&&... _Args) {
4.     // manages a callable object launched with supplied policy
5.     using _Ret    = _Invoke_result_t<decay_t<_Fty>, decay_t<_ArgTypes>...>;
6.     using _Ptype = typename _P_arg_type<_Ret>::type;
7.     //1 处
8.     _Promise<_Ptype> _Pr(
9.         _Get_associated_state<_Ret>(_Policy, _Fake_no_copy_callable_adapter<_Fty, _ArgT
   ypes...>(
10.                                     _STD forward<_Fty>(_Fnarg), _STD forward<_ArgTypes>(_Ar
   gs)...)));
11.     //2 处
12.     return future<_Ret>(_Pr._Get_state_for_future(), _Nil());
13. }
```

我们先看看 `_Get_associated_state` 的源码

```
1. template <class _Ret, class _Fty>
2. _Associated_state<typename _P_arg_type<_Ret>::type>* _Get_associated_state(
3.     launch _Psync, _Fty&& _Fnarg) { // construct associated asynchronous state object for th
   e launch type
4.     switch (_Psync) { // select launch type
5.     case launch::deferred:
6.         return new _Deferred_async_state<_Ret>(_STD forward<_Fty>(_Fnarg));
7.     case launch::async: // TRANSITION, fixed in vMajorNext, should create a new thread here
8.     default:
9.         return new _Task_async_state<_Ret>(_STD forward<_Fty>(_Fnarg));
10.     }
11. }
```

`_Get_associated_state` 做的事情很简单，根据我们不同的策略 `deferred` 还是 `async` 去构造不同的异步状态。如果是 `launch::async` 策略，我们创建一个 `_Task_async_state` 类型的指针对象，我们将这个指针转化为 `_Associated_state` 指针返回， `_Associated_state` 为 `_Task_async_state` 的基类。

`async` 内 1处用该返回值构造了一个 `_Promise<_Ptype>` 类型的对象 `_Pr`

`async` 内 2处 用 `_Pr._Get_state_for_future()` 返回值构造了 `future` ，该返回值是 `_State_manager<_Ty>` 类型对象。

因为 `future` 继承于 `_State_manager` ，所以 `_Pr._Get_state_for_future()` 返回的值主要用来构造future的基类。

析构 `future` 时要析构 `future` 子类再析构其基类， `future` 本身的析构没有什么，而其基类 `_State_manager<_Ty>` 析构时调用如下

```
1.  ~_State_manager() noexcept {
2.      if (_Assoc_state) {
3.          _Assoc_state->_Release();
4.      }
5.  }
```

看源码我们知道 `_Assoc_state` 是 `_Associated_state<_Ty> *` 类型

```
1.  _Associated_state<_Ty>* _Assoc_state;
```

`_Assoc_state *` 就是我们之前在 `_Get_associated_state` 中开辟并返回的 `_Task_async_state*` 类型转化的。

我们沿着 `_Assoc_state->_Release` 一路追踪，会发现最终调用了下面的代码

```
1. void _Delete_this() { // delete this object
2.      if (_Deleter) {
3.          _Deleter->_Delete(this);
4.      } else {
5.          delete this;
6.      }
7.  }
```

如果没有删除器则会直接调用 `delete this` ，会调用 `_Assoc_state` 的析构函数，因其析构函数

为虚析构，进而调用 _Task_async_state 的析构函数

所以我们 ~_State_manager() 调用的其实是 _Task_async_state 的析构函数, 我们看一下 _Task_async_state 的析构函数源码

```
1. virtual ~_Task_async_state() noexcept {
2.     _Wait();
3. }
4.
5. virtual void _Wait() override { // wait for completion
6.      _Task.wait();
7.  }
```

从源码中可以看到 _Task_async_state 被析构时会等待任务完成，这也就是future需等待任务完成后才析构的原因。

仅仅介绍这个事情不是我得初衷，我们介绍一种更为隐晦的死锁情况, 看下面的代码

```
1. void DeadLock() {
2.     std::mutex  mtx;
3.     std::cout << "DeadLock begin " << std::endl;
4.     std::lock_guard<std::mutex>  dklock(mtx);
5.     {
6.         std::future<void> futures = std::async(std::launch::async, [&mtx]() {
7.             std::cout << "std::async called " << std::endl;
8.             std::lock_guard<std::mutex>  dklock(mtx);
9.             std::cout << "async working...." << std::endl;
10.            });
11.    }
12.
13.    std::cout << "DeadLock end " << std::endl;
14. }
```

上面函数的作用意图在主线程中先执行加锁，再通过 async 启动一个线程异步执行任务，执行的任务与主线程互斥，所以在 lambda 表达式中加锁。
但是这么做会造成死锁，因为主线程输出”DeadLock begin “加锁，此时 async 启动一个线程，那么 lambda 表达式会先输出”std::async called “.
但是在子线程中无法加锁成功，因为主线程没有释放锁。而主线程无法释放锁，因为主线程要等待 async 执行完。
因为我们上面提到过， futures 处于局部作用域，即将析构，而析构又要等待任务完成，任

务需要加锁，所以永远完成不了，这样就死锁了。

所以使用 `async` 要注意其返回的future是不是shared state的最后持有者。

这里有个粉丝问道能不能用async实现这样的需求

1. 你的需求是func1 中要异步执行asyncFunc函数。
2. func2中先收集asyncFunc函数运行的结果，只有结果正确才执行
3. func1启动异步任务后继续执行，执行完直接退出不用等到asyncFunc运行完

如果我们理解了async的原理后不难实现如下代码

```
int asyncFunc() {
    std::this_thread::sleep_for(std::chrono::seconds(3));
    std::cout << "this is asyncFunc" << std::endl;
    return 0;
}

void func1(std::future<int>& future_ref) {
    std::cout << "this is func1" << std::endl;
    future_ref = std::async(std::launch::async, asyncFunc);
}

void func2(std::future<int>& future_ref) {
    std::cout << "this is func2" << std::endl;
    auto future_res = future_ref.get();
    if (future_res == 0) {
        std::cout << "get asyncFunc result success !" << std::endl;
    }
    else {
        std::cout << "get asyncFunc result failed !" << std::endl;
        return;
    }
}

//提供多种思路，这是第一种
void first_method() {
    std::future<int> future_tmp;
    func1(future_tmp);
    func2(future_tmp);
}
```

上面的例子我们保证在 `func1` 和 `func2` 使用的是 `future` 的引用即可。这样 `func1` 内不会因为启动 `async` 而阻塞，因为 `future_ref` 不是 `shared state` 最后持有者。

如果真的想实现一个纯异步的操作倒也不难，可以这样实现

```
template<typename Func, typename... Args >
auto ParallenExe(Func&& func, Args && ... args) -> std::future<decltype(func(args...))> {
    typedef decltype(func(args...)) RetType;
    std::function<RetType()> bind_func = std::bind(std::forward<Func>(func), std::forward<Args>(args)...);
    std::packaged_task<RetType()> task(bind_func);
    auto rt_future = task.get_future();
    std::thread t(std::move(task));

    t.detach();

    return rt_future;
}
```

上面的函数 `ParallenExe` 内部我们通过 `bind` 操作将函数和参数绑定，生成一个返回值为 `RetType` 类型，参数为 `void` 的函数 `bind_func` 。
接着我们用这个函数生成了一个 `packaged_task` 类型的对象 `task` ，这个 `task` 获取 `future` 留作以后函数结束时返回。
我们启动了一个线程处理这个 `task` ，并将这个线程 `detach` ，保证其分离独立运行。返回的 `rt_future` 并不是 `shared state` 最后持有者，因为 `task` 内部也会持有 `shared_state` ，引用计数并不会变为0，所以并不会触发如下析构

```
void _Release() { // decrement reference count and destroy when zero
    if (_MT_DECR(_Refs) == 0) {
        _Delete_this();
    }
}
```

那么我们写一个函数测试一下

```
void TestParallen1() {
    int i = 0;
    std::cout << "Begin TestParallen1 ..." << std::endl;
    {
        ParallenExe([](int i) {
            while (i < 3) {
                i++;
                std::cout << "ParllenExe thread func " << i << " times" << std::endl;
                std::this_thread::sleep_for(std::chrono::seconds(1));
            }
            }, i);
    }

    std::cout << "End TestParallen1 ..." << std::endl;
}
```

在上面的函数中我们有意让 `ParallenExe` 放在一个局部的 `{}` 中执行，意在局部作用域结束后 `ParallenExe` 返回的 `future` 引用计数 `-1` ,以此证明其引用计数是否为0，

如果引用计数为0，则会执行 `future` 的析构进而等待任务执行完成，那么看到的输出将是

```
Begin TestParallen1 ...
ParllenExe thread func 1 times
ParllenExe thread func 2 times
ParllenExe thread func 3 times
End TestParallen1 ...
```

如果引用计数不会为0，则不会执行 `future` 的析构函数，那么看到的输出是这样的

```
Begin TestParallen1 ...
End TestParallen1 ...
ParllenExe thread func 1 times
ParllenExe thread func 2 times
ParllenExe thread func 3 times
```

我们在 `main` 函数中调用做测试, 因为要防止主线程过早退出，所以我们先让主线程睡眠4秒

```
int main()
{
    TestParallen1();
    std::this_thread::sleep_for(std::chrono::seconds(4));
    std::cout << "Main Exited!\n";
}
```

而事实证明是第二种，输出如下

```
Begin TestParallen1 ...
End TestParallen1 ...
ParllenExe thread func 1 times
ParllenExe thread func 2 times
ParllenExe thread func 3 times
```

由此可见我们实现了不会阻塞的并发函数，但是也会存在一些顾虑，比如我们的主线程如果不睡眠4秒，那很可能主线程退出了子线程的任务没执行完而被强制回收。
所以归根结底，这种方式我们也需要在合适的时候等待汇合，比如调用 `future的get或wait` 操作

```
void TestParallen2() {
    int i = 0;
    std::cout << "Begin TestParallen2 ..." << std::endl;

    auto rt_future = ParallenExe([](int i) {
            while (i < 3) {
                i++;
                std::cout << "ParllenExe thread func " << i << " times" << std::endl;
                std::this_thread::sleep_for(std::chrono::seconds(1));
            }
            }, i);

    std::cout << "End TestParallen2 ..." << std::endl;

    rt_future.wait();
}
```

这就是我说的，归根结底C++和js的体系不一样，C++要管理开辟的线程生命周期，我们总归要在合适的时机汇合。
所以 `std::async` 会返回 `future` ， `future` 会判断是不是最后持有的 `shared_state` 进而帮我们

做汇合操作,这并不是缺陷而是安全性的保证。至于我们不想在该处汇合，只要保证该处 `future` 不会是最后持有 `shared_state` 的即可。

总结

本文介绍了 `thread` 和 `async` 的源码级分析，回答了大家常遇到的问题。

详细源码

https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day09-QASummary (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day09-QASummary)

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

浏览(1986) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(590) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)